Mike Wolak · mike@epromfoundry.com · (512) 944-8586

Midway A-21464 Seattle · R5000 150 MHz · Voodoo SST-1 · measured on the unit under test

SNES Emulator for the Midway Seattle Board — Snes9x 1.39

Port of the Snes9x 1.39 core to the Midway A-21464 arcade board (QED R5000 at 150 MHz, 8 MB DRAM, 3Dfx Voodoo SST-1 with one TMU, DCS2 ADSP-2115 sound). The 65C816 loop, the SPC700 loop, the tile renderers and the line fills are R5000 assembler; the frame is drawn as 8-bit palette indices and colored by the texture unit. Super Mario World runs at 60.0 emulated and 59.8 shown frames a second on the board.

SUMMARY

- Super Nintendo Entertainment System emulator for the Midway A-21464 board
- Snes9x 1.39 core with instruction loops and tile renderers in R5000 assembler
- 60.0 emulated frames per second, 59.8 shown, 150 MHz R5000 without second-level cache
- 31,250 Hz 16-bit stereo sound through the DCS2 ADSP-2115

KEY FEATURES

- 65C816 instruction loop: 2,306 lines of assembler, four register-width modes, 10-instruction dispatch
- SPC700 instruction loop: 1,452 lines of assembler, 216 of 256 opcodes, batch execution against recorded cycle limits
- Tile renderers for an RGB565 frame and for a frame of 8-bit palette indices; runs of background tiles in one call
- 11 instructions for a group of four pixels in the palette-index renderer, 28 in the RGB565 renderer
- 256 x 224 P8 texture by GT-64010 DMA, 256-entry palette in the texture unit, one textured quad per frame
- Line fills by cache-line allocation without a DRAM read
- Run-time selection of every assembler path against its C reference
- CRC-32 verification of machine state, rendered frames, displayed buffer and mixed sound

PERFORMANCE SUMMARY

ITEM	RESULT
Emulated frame rate, board	59.9 to 60.0 frames/s
Shown frame rate, board, 512 x 240	59.7 to 59.8 frames/s
Rendered frame, emulation and drawing	12.6 to 12.9 ms (budget 16.64 ms)
Sound mixing	1.8 ms a frame, 31,250 Hz stereo
Texture and quad	0.25 ms of CPU time a frame
Frame CRC against the C core	38 lines, 1,140 frames equal (board)
Game tested	Super Mario World (USA)
Other games, special chips	untested
Disk build	1,358,916 bytes, CRC32 6f5d6ab9

1 GENERAL DESCRIPTION

The emulator is the Snes9x 1.39 release source with a platform layer for the Midway Seattle board and a set of changes to the core. It is built into the disk build of the board's bring-up monitor and started from the launch menu of the boot ROM. Games are read from the `/snes` directory of an exFAT volume on the CompactFlash card; battery RAM is kept in `/saves`.

The board has a QED R5000 at 150 MHz with 32 KB primary caches and no second-level cache, 8 MB of DRAM behind a Galileo GT-64010 system controller, a 3Dfx Voodoo SST-1 with one texture unit and 4 MB of texture memory on a 33 MHz PCI bus, and the Midway DCS2 sound board with an ADSP-2115. The core in C renders 10 frames a second on this processor. The work recorded here replaces the inner loops of the core by R5000 assembler, changes the drawing path so that each pixel is written once, and moves the color lookup and the scaling of the frame to the Voodoo.

2 FUNCTIONAL BLOCK DIAGRAM

```
                      +------------------------------------------------------------+
                      | R5000, 150 MHz, 32 KB + 32 KB primary cache, 8 MB DRAM     |
                      |                                                            |
CompactFlash, exFAT   |  65C816 loop (cpu65.S) --cycle limits--> SPC700 loop       |
/snes   images   ---> |                                          (spc700.S)        |
/saves  battery  <--> |  PPU state, HDMA, DMA (Snes9x 1.39, C)                     |
                      |        |                                                   |
JAMMA controls   ---> |        v                                                   |
                      |  tile renderers (tile8.S, tile16.S), line fills (draw.S)   |
                      |        |                                                   |
                      |        v                                                   |
                      |  frame: 8-bit palette indices, 57,344 bytes  ----------+   |
                      |                                                        |   |
                      |  S-DSP mixer (C), 31,250 Hz stereo  ----+              |   |
                      +-----------------------------------------|--------------|---+
                                                                |              |
                                                                v              v
                          DCS2 FIFO, ADSP-2115 stream program, DAC    GT-64010 DMA channel 0
                                                                      256 bytes per line, PCI
                                                                               |
                                                                               v
                                          Voodoo SST-1: P8 texture 256 x 224, 256-entry palette,
                                          one textured quad per frame, swap at retrace
```

3 SPECIFICATIONS

A-21464 board, boot ROM `seattle_monitor23_27C4001.bin`, release disk build, Super Mario World (USA) attract demo, sound on, automatic frame skip, texture by DMA, 512 x 240 unless noted. Minimum and maximum over the 10-s reports of the runs of the Board Measurements section.

PARAMETER	CONDITIONS	MIN	MAX	UNIT
Emulated frame rate		59.9	60.0	frames/s
Shown frame rate	512 x 240, 59.98 Hz	59.7	59.8	frames/s
Shown frame rate	512 x 256, 54 Hz	53.6	53.7	frames/s
Rendered frame time	emulation and drawing	12,398	12,907	us
Skipped frame time	emulation alone	5,731	6,400	us
Sound mixing time	per emulated frame	1,772	1,819	us
Texture transfer start	CPU time per shown frame	250	254	us
Idle time	per emulated frame, 512 x 240	382	401	us
Texture transfers incomplete after 100 ms		0	0	
Sound sample rate	stereo, 16 bit	31,250	31,250	Hz
Arena free at game start	image buffer 0x260000 bytes	19	23	KB
Disk build size	payload	1,358,916	1,358,916	bytes

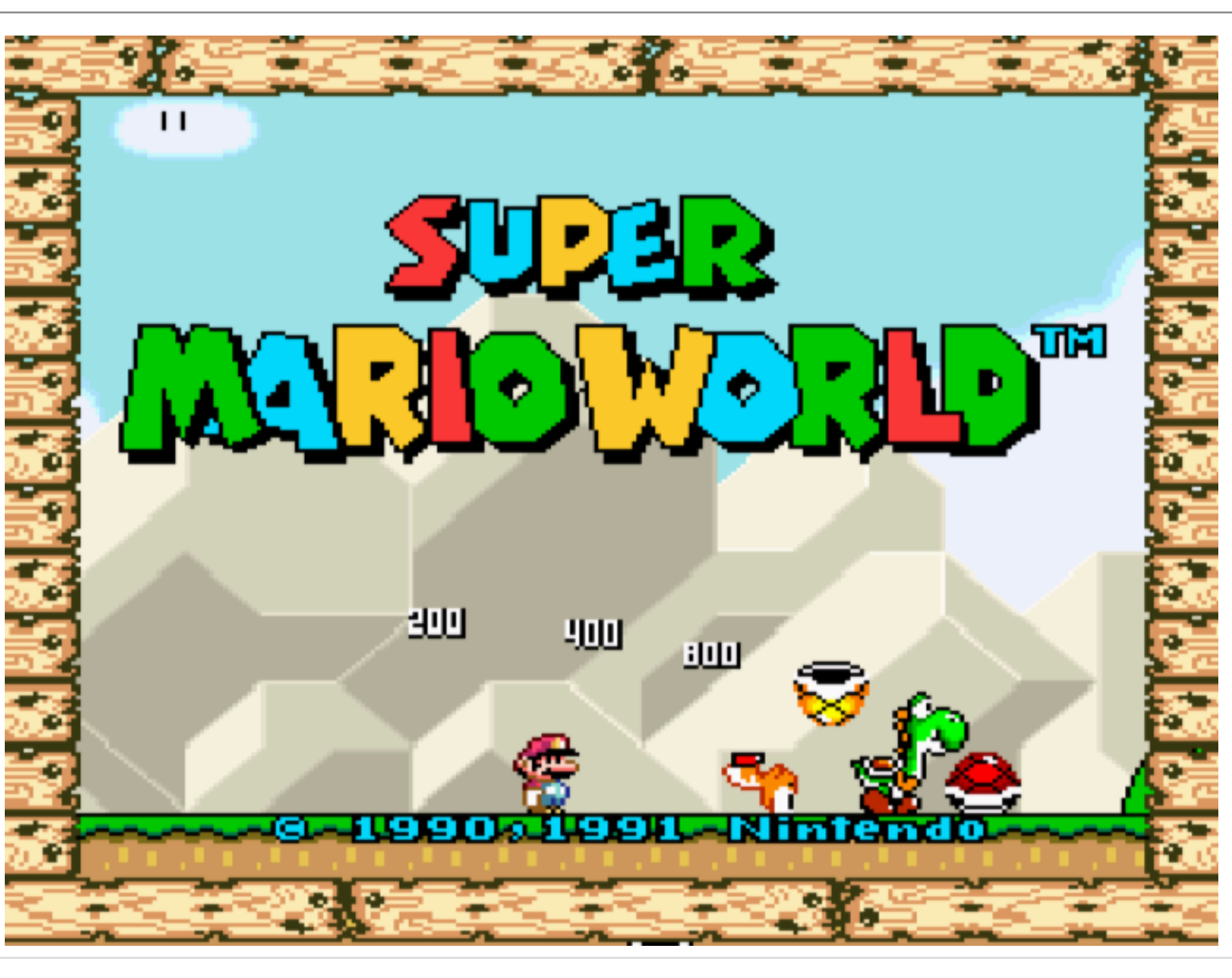

Figure 1. Super Mario World (USA), attract demo: sprites and score digits over three background layers, frame of 8-bit palette indices displayed through the TMU palette. MAME 0.289 driver blitz99, snapshot of the emulated Voodoo output in the 512 x 240 mode, shown at the 4:3 proportion of the monitor. The program has no screen capture path on the board; the picture on the board was confirmed correct by the owner at the cabinet.

4 FEATURES

SYSTEM	VIDEO AND SOUND
Snes9x 1.39 core: 65C816, SPC700, S-DSP, PPU modes 0 to 7, HDMA, color math	Frame 256 x 224 or 239, drawn as RGB565 or as 8-bit palette indices
65C816 loop `cpu65.S`, SPC700 loop `spc700.S`, both selectable against the C loops at run time	Texture formats RGB565 and P8, 256 texels a row, written by GT-64010 DMA channel 0
Tile renderers `tile16.S` (RGB565) and `tile8.S` (palette indices), selectable against the C renderer	One textured quad a frame, point or bilinear filter, fill, integer or 1:1 scaling
Renders in strips of 16 lines that stay in the 32 KB data cache	Video modes 512 x 240 (59.98 Hz, default), 512 x 256 (54 Hz), 384 x 240, 512 x 384
Automatic frame skip with a lateness tolerance	Sound: 31,250 Hz stereo 16-bit to the DCS2 FIFO through an ADSP-2115 stream program
Game list from `/snes` on exFAT (FatFs R0.16), battery RAM in `/saves`	JAMMA controls mapped to one or two SNES pads
State and frame CRC lines for verification (`check`, `fcrc`, `vcrc`, `scrc`)	1 kHz program-counter profile with 128-byte and 8-byte buckets

5 BASIS OF THE PORT

ITEM	VALUE
Upstream project	Snes9x 1.39 release source `s9xs139.zip`, <https://www.lysator.liu.se/snes9x/1.39/s9xs139.zip>, 666,463 bytes
Upstream SHA-256	`996b3ed10debca64e02e31f28e3e7f78b3d39608aa918cb739b2c2fa17bce27b`
Authors	Gary Henderson, Jerremy Koot; Super FX, DSP-1 and C4 code by the contributors named in `copyright.h`
License	Snes9x license: use, copy, modify and distribute for non-commercial purposes
Core changes	`patches/snes9x-1.39-seattle.patch`: applied to the unpacked archive it reproduces the build tree
Platform layer	`snes/`: `s9x_seattle.c`, `s9x_cpu.cpp`, `s9x_offs.cpp`, `s9x_port.h`, `menu.c`, and the assembler files of the next section
Host tree	bring-up monitor of the Blitz99 project, repository seattle-quake2, branch `bringup-rom`, commit `de321c5`
Palette download	register sequence of `_grTexDownloadPalette`, Glide 2.x, 3Dfx Interactive
File system	FatFs R0.16, ChaN

6 ASSEMBLER ROUTINES

FILE	LINES	ROUTINE	REPLACES
`cpu65.S`	2,306	`s9xa_run`	`S9xMainLoop` of `cpuexec.cpp` with the opcode handlers of `cpuops.cpp`
`spc700.S`	1,452	`s9xa_spc_sync`	the `APU_EXECUTE` loop and the opcode functions of `spc700.cpp`
`draw.S`	75	`s9x_fill_lines`, `s9x_copy_lines`	word-store loops and `memset` of the depth and back-color line fills
`blit.S`	94	`snes_blit_asm`	the 1:1 copy of the frame to the linear frame buffer (scaling 1:1 only)
`tile16.S`	589	`s9x_tile16`, `s9x_ctile16`, `s9x_tilerun16`	`DrawTile16`, `DrawClippedTile16` of `tile.cpp`, the middle loop of `DrawBackground`
`tile8.S`	521	`s9x_tile8`, `s9x_ctile8`, `s9x_tilerun8`	the same three for a frame of 8-bit palette indices

All routines are o32, little-endian, MIPS III instructions with the R5000 cache operations. Integer registers hold 32-bit values only: the interrupt entry of the host tree saves 32 bits of each register, so a 64-bit integer value does not survive an interrupt. Structure offsets of the core come from `s9x_offs.h`, generated by compiling `s9x_offs.cpp` to assembler text.

7 PERFORMANCE BY OPTIMIZATION STEP

Board, Super Mario World attract demo, sound on, automatic frame skip, 10-s reports. A rendered frame is emulation plus drawing; a skipped frame is emulation alone. Each row adds its step to the rows above.

STEP	KIND	RENDERED FRAME	SKIPPED FRAME	SHOWN FRAMES/S
Unmodified core, C loops	reference	36.4 ms	9.5 ms	10.2
65C816 loop `cpu65.S`	assembler	33.6 ms	6.4 ms	16.4
SPC700 loop `spc700.S` with ring scheduling	assembler	32.9 ms	5.7 ms	17.9
Tile renderer with depth words, backdrop passes, sub screen drawn into the main screen, strips; line fills `draw.S`	C and assembler	16.9 ms	5.6 ms	36.5
Frame texture by GT-64010 DMA	hardware	17.4 ms	5.9 ms	39.3
`tile16.S`, `tile8.S`, P8 texture, 512 x 240 mode	assembler and hardware	12.6 ms	6.3 ms	54.7
Frame skip with lateness tolerance	pacing	12.6 ms	5.8 ms	59.8

The DMA row shows a longer rendered frame because the texture transfer then overlaps the emulation of the next frame; the video time on the CPU falls from 2.6 ms to 0.5 ms in the same step. The last assembler row was measured on the board as one step. Its parts were measured separately in MAME, whose times follow the instruction count:

STEP, MAME, FRAMESKIP 1, SOUND OFF	RENDERED FRAME
C tile renderer	12,768 us
`s9x_tile16` and `s9x_ctile16`	11,652 us
plus `s9x_tilerun16`	10,553 us
plus delay slots and sign tests in the group and pixel code	10,251 us
plus the frame of palette indices, `tile8.S`	8,724 us

8 65C816 LOOP (CPU65.S)

ITEM	IMPLEMENTATION
Register allocation	program counter as a host pointer, cycle count, A, X, Y, the zero, negative and carry values, the base of `Registers` and of `CPU`, the data bank, the memory speed and the jump table of the current mode stay in registers across opcodes
Dispatch	one jump table of 256 entries for each of the four M and X width modes; 10 instructions from the end of one opcode to the first of the next
Loop limit	one comparison of the cycle count with the next event; H-blank processing and interrupts run only when it fails
Memory	RAM and ROM blocks through `Memory.Map` inline; bank 0 addresses below 0x2000 through a fixed base; I/O through the core's C functions
Opcodes left to the C handlers	BRK, COP, RTI, WAI, STP, XCE, SED, REP and SEP in emulation mode, ADC and SBC in decimal mode, branches while the SPC700 is off
Exactness	every handler makes the memory accesses of the C handler in the same order with the same cycle additions

```
next:                                   # after each opcode
    slt     at, CYC, LIM                # cycle count below the next event?
    beqz    at, slow                    # no: H-blank check and interrupt step
    lbu     v0, 0(PC)                   # (delay slot) the opcode byte
    sw      CYC, 0(RP)                  # record this pass's cycle count for the SPC700
    addiu   RP, RP, 4
disp:
    move    OPS, PC                     # CPU.PCAtOpcodeStart
    sll     v1, v0, 2                   # opcode x 4
    addu    v1, v1, TAB                 # entry in the table of the M/X mode
    lw      v1, 0(v1)
    addiu   PC, PC, 1
    jr      v1                          # the handler
    addu    CYC, CYC, MS                # (delay slot) the opcode fetch cycles
```

MEASUREMENT, BOARD	C LOOP	`CPU65.S`
Skipped frame	9,350 / 10,293 / 9,274 us	6,357 / 6,808 / 6,228 us
Rendered frame	36,435 / 37,365 / 30,454 us	33,567 / 34,045 / 25,559 us
Share of the profile samples, 65C816 and SPC700 dispatch	42.1 %	23.9 %
Shown frames/s	10.4 / 8.2 / 13.9	16.4 / 15.2 / 22.6

Correctness: 126 `check` lines (frames 60 to 7,560) equal to the C loop in MAME; 10 lines equal on the board and equal to MAME. The game executes 6,700 65C816 opcodes an emulated frame, one of them by a C handler.

9 SPC700 LOOP (SPC700.S)

The core runs the SPC700 up to the 65C816 cycle count before every 65C816 opcode. `cpu65.S` instead stores the cycle count of each pass in a ring (the two instructions marked in the listing above) and runs the SPC700 over the recorded limits before every call of C code, before the idle-loop shutdown and at the end of the frame. The two processors exchange data only through C code, so both execute the same opcodes against the same values as in the core.

ITEM	IMPLEMENTATION
Register allocation	program counter as a host pointer, A, X, Y, S, the zero and carry values, the direct page, the cycle count, the limit and the RAM base stay in registers within a batch
Opcodes in assembler	216 of 256; DIV, DAA, DAS, BRK, RETI, SLEEP, STOP, TCALL, TSET1, TCLR1, the bit operations on absolute addresses and JMP [abs+X] run the core's function
Memory	RAM inline; the registers 0xF0 to 0xFF with the port, counter and timer rules inline; DSP and control registers through the core's C functions
Exactness of the batch	while the SPC700 stays running, a run against each recorded limit in turn equals a run against the last one; an opcode that stops it ends the batch at the limit that opcode ran under

```
spc_next:
    slt     at, ALIM, ACY               # APU.Cycles above the limit: the batch is done
    bnez    at, spc_exit
spc_go:
    lbu     v0, 0(SPC)                  # (delay slot) the opcode byte
    sll     v1, v0, 2
    sw      ACY, 24($sp)                # cycle count at the opcode's start
    lui     a0, %hi(S9xAPUCycles)
    addu    a0, a0, v1
    lw      a0, %lo(S9xAPUCycles)(a0)   # cycles of this opcode
    lui     a1, %hi(s9xa_spc_tables)
    addu    a1, a1, v1
    lw      a1, %lo(s9xa_spc_tables)(a1)
    jr      a1                          # the handler
    addu    ACY, ACY, a0                # (delay slot) APU.Cycles += S9xAPUCycles [opcode]
```

MEASUREMENT, BOARD	65C816 IN ASSEMBLER, SPC700 BEFORE EACH OPCODE IN C	BOTH IN ASSEMBLER
Skipped frame	6,357 / 6,808 / 6,228 us	5,653 / 6,052 / 5,594 us
Both loops, time an emulated frame	8.1 ms with both in C	4.3 ms
Shown frames/s	16.4 / 15.2 / 22.6	17.9 / 16.7 / 23.8

Correctness: 126 `check` lines including the CRC-32 of the SPC700 RAM equal to the C loops in MAME; 9,598,822 SPC700 opcodes executed in 50 s, the count of the C functions.

10 LINE FILLS (DRAW.S)

Each line of the screen and of the depth buffer is filled before the tiles are drawn. A store to a line that is not in the data cache makes the R5000 read that line from DRAM first. The fill routines allocate each 32-byte line with the Create Dirty Exclusive cache operation, which marks the line valid and dirty without the read, and then write all of it.

```
s9x_fill_lines:                         # a0 destination (32-byte aligned), a1 word, a2 bytes
    beqz    $a2, 2f
    addu    $a2, $a0, $a2               # (delay slot) end address
1:  cache   0x0d, 0($a0)                # Create Dirty Exclusive: the line without a DRAM read
    sw      $a1, 0($a0)
    sw      $a1, 4($a0)
    sw      $a1, 8($a0)
    sw      $a1, 12($a0)
    sw      $a1, 16($a0)
    sw      $a1, 20($a0)
    sw      $a1, 24($a0)
    addiu   $a0, $a0, 32
    bne     $a0, $a2, 1b
    sw      $a1, -4($a0)                # (delay slot) the eighth word
2:  jr      $ra
    nop
```

Measured alone on the board the fills change the rendered frame by less than the variation between runs (25,822 against 25,912 us). They are part of the drawing steps that follow: with the sub screen drawn into the main screen, the backdrop value is filled first and no backdrop pass reads the frame again (21,305 to 19,572 us). `blit.S` uses the same cache operation with Hit Writeback Invalidate to send a line to the frame buffer as one 8-word PCI write; it serves the 1:1 scaling only.

11 TILE RENDERER FOR RGB565 (TILE16.S)

A tile row is two words of the tile cache, one pixel value in each byte. The four depth bytes of a group of four pixels are compared with the depth of the layer in one word: byte k of ((Z1 - 1) x 0x01010101 | 0x80808080) minus the depth word has bit 7 set exactly when depth byte k is below Z1. A group with no such byte is skipped. A group with all four and no pixel of value 0 is written without a test per pixel. `ConvertTile` marks a tile that has no pixel of value 0, and such a tile takes a row loop without the zero test.

```
    lwl     $t2, 3($a1)                 # four depth bytes, unaligned
    lwr     $t2, 0($a1)
    subu    $t2, $t6, $t2               # t6 = (Z1 - 1) x 0x01010101 | 0x80808080
    and     $t2, $t2, $t8               # bit 7 of each byte: depth below Z1
    bne     $t2, $t8, group_m           # not all four: per pixel, or skip
    sll     $v0, $t0, 1                 # (delay slot) pixel 0 x 2, the palette offset
    andi    $v0, $v0, 0x1fe
    addu    $v0, $t5, $v0               # t5 = the palette of the tile
    srl     $v1, $t0, 7                 # pixel 1 x 2
    andi    $v1, $v1, 0x1fe
    lhu     $v0, 0($v0)                 # color of pixel 0
    addu    $v1, $t5, $v1
    srl     $t2, $t0, 15                # pixel 2 x 2
    andi    $t2, $t2, 0x1fe
    lhu     $v1, 0($v1)
    sh      $v0, 0($a0)                 # pixel 0 to the screen
    addu    $t2, $t5, $t2
    srl     $v0, $t0, 23                # pixel 3 x 2
    andi    $v0, $v0, 0x1fe
    lhu     $t2, 0($t2)
    sh      $v1, 2($a0)
    addu    $v0, $t5, $v0
    lhu     $v0, 0($v0)
    sh      $t2, 4($a0)
    swl     $t7, 3($a1)                 # four depth bytes = Z2, one unaligned word
    swr     $t7, 0($a1)
    b       group_n
    sh      $v0, 6($a0)                 # (delay slot) pixel 3
```

The group costs 28 instructions for four pixels in the row loop of a tile without a pixel 0. `s9x_tilerun16` draws a run of unclipped 8 x 8 tiles of one background row in one call. The values that the C code derives for every tile from the `BG` and `GFX` structures are read once per run. A blank tile, about half of the 4,000 tile calls of a frame of the attract demo, is passed over after 20 instructions:

```
r_next:
    beq     $s0, $s1, r_done            # end of the run
    lw      $t0, R_SHIFT($sp)           # (delay slot) BG.TileShift, kept in the frame
    lhu     $a0, 0($s0)                 # the tile map word
    lw      $t1, R_TADDR($sp)
    addiu   $s0, $s0, 2
    andi    $v0, $a0, 0x3ff             # tile number field
    sllv    $v0, $v0, $t0
    addu    $v0, $v0, $t1
    andi    $v0, $v0, 0xffff            # address of the tile in VRAM
    lw      $t1, R_BUFFERED($sp)
    srlv    $v0, $v0, $t0               # index of the tile in the tile cache
    addu    $t1, $t1, $v0
    lbu     $v1, 0($t1)                 # state: 0 not converted, 2 blank, 3 no pixel 0
    lw      $t2, R_PMASK($sp)
    beqz    $v1, r_core                 # not converted: the core converts and draws it
    xori    $t1, $v1, TILE_BLANK
    beqz    $t1, r_skip                 # blank: the next tile
    srl     $t5, $a0, 10                # (delay slot) palette field
```

Tile statistics of the attract demo, average of a rendered frame: 4,000 tile calls, 2,100 of them for blank tiles; 10,700 tile rows; 12,600 groups written whole, 5,900 skipped by the depth test, 1,700 written per pixel.

12 TILE RENDERER FOR PALETTE INDICES (TILE8.S)

In a frame of palette indices each pixel is one byte: the pixel value plus the palette base of the tile, which is the index of the color that the RGB565 renderer reads from the color table. The sum is below 256 for every tile of 2 or 4 bits a pixel, so the four pixels of a group are formed by one addition and stored as one word. The four table lookups and four halfword stores of the listing above are not executed. A group written in part is merged under a byte mask without a branch per pixel.

```
    lwl     $t2, 3($a1)                 # four depth bytes
    lwr     $t2, 0($a1)
    subu    $t2, $t6, $t2
    and     $t2, $t2, $t8               # bit 7 of each byte: depth below Z1
    bne     $t2, $t8, group_m
    addu    $v0, $t0, $t5               # (delay slot) four pixels + palette base x 0x01010101
    swl     $v0, 3($a0)                 # four screen bytes
    swr     $v0, 0($a0)
    swl     $t7, 3($a1)                 # four depth bytes = Z2
    b       group_n
    swr     $t7, 0($a1)                 # (delay slot)
group_m:
    beqz    $t2, group_n                # no byte below Z1: nothing
    srl     $v1, $t2, 7                 # (delay slot)
    subu    $v1, $t2, $v1               # 0x80 becomes 0x7f in each selected byte
    or      $v1, $v1, $t2               # 0xff in each byte to write
    lwl     $at, 3($a0)                 # the screen word
    lwr     $at, 0($a0)
    xor     $v0, $v0, $at
    and     $v0, $v0, $v1
    xor     $v0, $v0, $at               # new bytes where the mask is set
    swl     $v0, 3($a0)
    swr     $v0, 0($a0)                 # the depth word is merged the same way
```

A group written whole costs 11 instructions against 28. For a tile with pixels of value 0 the mask also excludes them: bit 7 of ((w and 0x7f7f7f7f) + 0x7f7f7f7f) or w is set in each byte of w that is not 0.

ITEM	IMPLEMENTATION
Lines drawn as indices	BG modes 0 and 1 without mosaic, with no color math in operation or with addition of the sub screen to the backdrop only
Entry 16	pixel value 0 of a palette, never drawn by a tile; holds the fixed color or black for the frame
Colors of a frame	the 255 colors and entry 16 at the first lines of the frame
Change within a frame	of a color, of entry 16, of the BG mode or of the color math: the lines drawn are expanded to RGB565 and the frame continues as RGB565; the next 60 rendered frames are RGB565 from their first line
Frame buffer	256 bytes a line, 57,344 bytes a frame of 224 lines; the RGB565 frame is 114,688 bytes

13 VIDEO PATH

ITEM	IMPLEMENTATION
Texture	256 texels a row at texture base 0, level 0 only; P8 (format 5) for a frame of palette indices, RGB565 (format 10) otherwise
Download of P8	textureMode bit 31 set (sequential 8-bit download): 64 consecutive words a row at byte t x 512 of the texture window; the bit is cleared before the quad
Transfer	GT-64010 DMA channel 0, chained records: one record of 256 bytes a line for P8, records of 32,768 bytes for RGB565; the frame's cache lines are written back before the start
Palette	256 writes to NCC table 0 registers I0 to Q3 with bit 31 set, entry index in bits 30 to 24 and the register number; RGB565 widened to 8 bits a component by bit replication; written when it differs from the palette last written
Quad	two triangles over the screen or over an integer multiple of the frame height, s 0 to 256, t 0 to the frame height; the swap is queued at the vertical retrace
Frame pacing	the program waits for the previous swap before it starts the next transfer, so it runs in step with the retrace. With automatic frame skip a frame is skipped only when the schedule is more than 75 % of a frame time behind
Rates	display 59.98 Hz in 512 x 240; SNES 60.1 Hz; about one frame in 8 s is skipped

14 BOARD MEASUREMENTS

A-21464 with the boot ROM `seattle_monitor23_27C4001.bin` in U32, Super Mario World (USA) attract demo, sound on, automatic frame skip, texture by DMA, 10-s reports 2 and 3.

RUN	EMULATED FRAMES/S	SHOWN FRAMES/S	RENDERED FRAME	SKIPPED FRAME	MIXING	VIDEO	IDLE
Before `tile16.S` and `tile8.S`, 512 x 256	59.9 / 60.0	39.3 / 39.3	17,102 / 17,384 us	5,524 / 5,892 us	1,959 / 2,008 us	509 / 517 us	1,198 / 847 us
Release build, 512 x 256 (54 Hz)	59.9 / 59.9	53.6 / 53.7	12,398 / 12,721 us	5,731 / 6,087 us	1,772 / 1,814 us	805 / 655 us	2,440 / 2,177 us
Release build, 512 x 240, frame skipped at any delay	60.0 / 59.9	54.7 / 55.0	12,553 / 12,902 us	6,297 / 6,336 us	1,772 / 1,815 us	736 / 779 us	2,162 / 1,722 us
Release build, 512 x 240, lateness tolerance 75 %	60.0 / 59.9	59.8 / 59.7	12,589 / 12,907 us	5,844 / 6,400 us	1,777 / 1,819 us	1,867 / 1,530 us	401 / 382 us

Every shown frame of the release build is a frame of palette indices (598 of 598 in a report). No texture transfer was incomplete after 100 ms. The video time of the last row includes the wait for the previous swap, 1,276 to 1,613 us.

15 VERIFICATION

SETTING	LINE PRINTED	CONTENT
`check N`	every N emulated frames	CRC-32 of work RAM, video RAM, the I/O register image and the SPC700 RAM; the 65C816 and SPC700 registers and cycle counters
`fcrc N`	every N emulated frames	CRC-32 chained over the RGB565 value of every pixel of every rendered frame
`vcrc N`	every N shown frames	CRC-32 of the displayed Voodoo buffer read back through the linear frame buffer
`scrc N`	every N emulated frames	CRC-32 chained over the mixed sound samples

TEST	RESULT
MAME, 120 s, each step of the tile work against the C renderer	98 `fcrc` lines (2,940 rendered frames) and 98 `check` lines equal
MAME, 70 s, displayed buffer, palette indices against RGB565	67 `vcrc` lines equal
MAME, 60 s, every frame changed over to RGB565 at line 112	38 `fcrc` and 38 `check` lines equal to the reference
Board, 40 s, release build against the MAME run of the C renderer	38 `fcrc` lines (1,140 rendered frames) and 38 `check` lines equal
Board, picture and sound at the cabinet	correct (owner)

16 BUILD AND INSTALLATION

1. Unpack `s9xs139.zip` into `$HOME/snes9x-1.39` and apply `patches/snes9x-1.39-seattle.patch` with `patch -p1`.
2. Place `snes/` as `bringup/snes/` in the bring-up tree of repository seattle-quake2 at commit `de321c5` or later; a tree without the SNES block takes `patches/bringup-snes-integration.patch`.
3. Build the disk build: `make -C bringup FATFS=1 BUILD=build-fs EXTRA="-DUART_DIV=0x00u" build-fs/payload.bin`. Toolchain: `mipsel-none-elf` GCC 12.2.0 with newlib.
4. Write the payload with its header sector to the card at LBA 0x3EE080 (monitor command `dsave`, or the remote protocol's `disk write`).
5. Copy game images (`.smc`, `.sfc`, `.swc`, `.fig`) to `/snes` on the exFAT volume of the card.
6. Start the program from the launch menu of the boot ROM: Other Apps, SNES.

17 SETTINGS

File `/snes/snes9x.cfg`, one `name value` a line. The settings that select the optimized paths:

NAME	VALUES	DEFAULT	EFFECT
`cpu`	asm, c	asm	65C816 loop `cpu65.S` or `S9xMainLoop`
`spc`	asm, c	asm	SPC700 opcodes in `spc700.S` or by the core's functions
`tile`	asm, c	asm	tile renderers of `tile16.S` or of `tile.cpp`
`pal8`	on, off	on	frames of 8-bit palette indices
`fill`	cde, cpu	cde	line fills by cache-line allocation or by word stores
`texture`	dma, cpu	dma	frame texture by GT-64010 DMA or by CPU block writes
`strip`	N	16	lines a strip of the renderer
`vmode`	0 to 4	0	video mode; 0: the mode of the ROM menu, 512 x 240 when it names none
`late`	0 to 400	75	percent of a frame time the schedule may be behind before a frame is skipped
`frameskip`	auto, 0 to 9	auto	automatic skip or a fixed number of skipped frames

The complete list is in `docs/SNES.md` section 6.

18 LIMITS AND UNTESTED ITEMS

ITEM	STATE
Games other than Super Mario World (USA)	untested
Super FX, SA-1, DSP-1, C4, S-DD1, HiROM images, PAL timing	untested
Frame of palette indices in BG mode 0, with 16 x 16 tiles, with color windows	untested
BG modes 2 to 7, mosaic, color math on a layer	drawn as RGB565 by the C renderer; speed not measured
P8 texture with the bilinear filter, texture by CPU block writes	untested
Memory	19 KB of the arena free at game start with a 2.4 MB image buffer; a second game start without a reset failed once to allocate, cause not established
Compiler options	`-O3` for the core enlarges the image by 123 KB and the graphics tables no longer fit the arena
Screen capture on the board	not available in the program; Figure 1 is a MAME snapshot

Platform layer, assembler loops, core changes and this document by Mike Wolak, EPROM Foundry Inc., project Blitz99 (repositories seattle-snes and seattle-quake2). Snes9x 1.39 is the work of Gary Henderson, Jerremy Koot and the contributors named in its copyright.h and is distributed under the Snes9x license (non-commercial use). The palette download sequence follows _grTexDownloadPalette of Glide 2.x, 3Dfx Interactive. Super Mario World is a work of Nintendo; no game image is distributed. Midway and NFL Blitz are marks of their owners.